Keyword Extraction from Scanned Academic Notices using TF-IDF and KeyBERT

MLLP Mini Project · Experiential Learning Group Assignment · Task 13

12 November 2026

1. Title and Group Details

Project title: Keyword Extraction from Scanned Academic Notices using TF-IDF and KeyBERT

Course: Machine Learning and Language Processing (MLLP)

Name	Roll number	Contribution
Karan Buragohain	2304024	Coding, Implementation, Github and Datacuration
Bikash Pradhan	2304020	Model Traning, Datacuration, Report
Ayushmaan Nath	2304019	GitHub, Matplotlib, PPT, Video Presentation

2. Problem Statement and Objectives

Colleges publish most of their notices and circulars as scanned images or PDFs. A scan is just pixels, so nobody can search it, index it or filter it by topic. If a student wants the notice about fees or exams, they have to open and read each one.

This project builds a system that takes a scanned notice image and returns its most important keywords. Those keywords can then be used to search, tag or route notices automatically.

Motivation

- Notice boards and college websites hold hundreds of scanned circulars that a computer can't read.
- Keywords give a short, searchable summary of each notice, for example *hall ticket, end semester, timetable*.
- The task joins computer vision (OCR) with NLP, which fits the course theme of language processing for images and text.

Objectives

1. Convert scanned notices to text using image preprocessing and Tesseract OCR.
2. Extract keywords with two methods: a statistical one (TF-IDF) and a semantic, transformer-based one (KeyBERT).
3. Evaluate both methods against labelled ground-truth keywords using precision, recall and F1.
4. Build a simple web demo where anyone can upload a notice and see its keywords.

Expected outcomes: a working end-to-end pipeline, a numerical comparison of TF-IDF and KeyBERT, and an interactive Gradio app.

3. Proposed Methodology and Processing Pipeline

The system is a six-stage pipeline. A scanned image goes in, OCR turns it into text, the text is cleaned, and two keyword extractors run side by side on it. Their output is then scored against the labelled keywords. Figure 1 shows the whole pipeline.

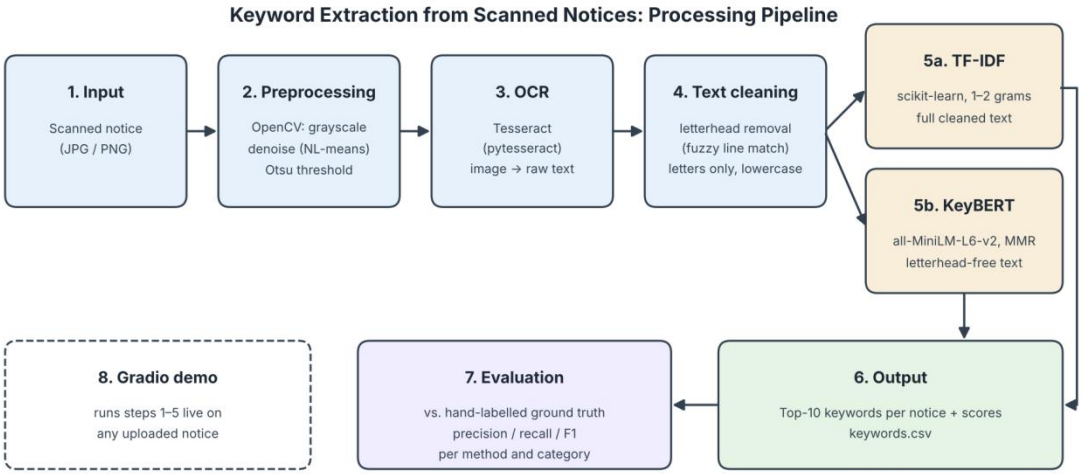


Figure 1: Processing pipeline (block diagram).

Step-by-step stages

1. **Input.** A scanned academic notice as a JPG or PNG image. The dataset has 12 notices from one institute, split across four categories (exam, fee, holiday and event) with 3 notices each.
2. **Preprocessing (OpenCV).** The image is converted to grayscale, denoised with non-local means denoising and binarised with Otsu's threshold. This hands Tesseract clean black text on a white background.
3. **OCR (Tesseract).** pytesseract reads the binarised image and returns the raw text line by line.
4. **Text cleaning.**
   - *Letterhead removal:* any line that shows up in at least half of the notices (institute name, address, approval line, footer) is treated as letterhead. Lines that match it with at least 85% similarity are removed, along with the reference-number line. Fuzzy matching is needed because OCR makes small mistakes, such as "Copy ta" for "Copy to".
   - *Normalisation:* only letters are kept, extra spaces are collapsed and everything is lowercased.
5. **Keyword extraction.**
   - *TF-IDF* runs on the full cleaned text. Inverse document frequency already gives low weight to text that repeats in every notice, so the letterhead does little harm here.
   - *KeyBERT* runs on the letterhead-free text, because an embedding model treats repeated letterhead as central to the document.
6. **Output and evaluation.** The top 10 keywords from each method are saved to keywords.csv and compared with the ground truth.

Input, intermediate processing and output

Stage	Data	Example (notice1.jpg)
Input	Scanned image, 1240 × 1754 px	End Semester Examination notice
After OCR	Raw text, about 800 characters	"END SEMESTER EXAMINATION - TIMETABLE To: All B.Tech Students …"
After cleaning	Lowercase body text	"end semester examination timetable to all b tech students …"

Stage	Data	Example (notice1.jpg)
Output	Top-10 keywords per method	KeyBERT: examination timetable, semester examinations, examination hall

4. Models and Tools Used

The pipeline uses one pretrained transformer (all-MiniLM-L6-v2, inside KeyBERT) and one classical statistical model (TF-IDF). Tesseract handles the OCR.

Component	Model / library	Role	Source
OCR engine	Tesseract OCR (LSTM engine), via pytesseract	Image to text	tesseract-ocr/tesseract
Image preprocessing	OpenCV (opencv-python-headless)	Grayscale, denoise, Otsu threshold	opencv.org
Statistical keywords	TF-IDF, scikit-learn TfidfVectorizer (unigrams + bigrams, English stopwords)	Terms frequent in one notice but rare across notices	scikit-learn docs
Semantic keywords	KeyBERT with Maximal Marginal Relevance (diversity 0.3)	Phrases whose embedding is closest to the whole notice	MaartenGr/KeyBERT
Embedding model	sentence-transformers/all-MiniLM-L6-v2 (384-dim, 22M parameters)	Sentence embeddings for KeyBERT	Hugging Face
Evaluation and data	pandas, scikit-learn stopword list, matplotlib	Scoring, tables, charts	pandas.pydata.org
Web demo	Gradio	Upload a notice, see OCR text and keywords	gradio.app

How KeyBERT works: it embeds the whole notice and every candidate phrase of one or two words using MiniLM, then ranks the phrases by cosine similarity to the notice. MMR then gives up a little similarity in return for diversity, so near-duplicate phrases are not all picked.

How TF-IDF works: a term's score is its frequency in the notice multiplied by the log of how rare it is across all 12 notices. Words found everywhere, like the institute name, end up with low scores.

Environment: Google Colab (Python 3, CPU runtime). Nothing is trained and no GPU is needed, since all models are used pretrained.

5. Implementation and Usability

The whole system lives in one Google Colab notebook that runs top to bottom in a few minutes on a free CPU runtime.

Implementation procedure

1. **Dataset creation.** We recreated 12 notices in the style of real college circulars, each with a letterhead, reference number, date, title, body and signatory. They were not photographed from a notice board. There are 3 notices in each of the 4 categories (exam, fee, holiday, event).
2. **Ground-truth labelling.** For each notice we wrote 6–7 keywords that a reader would plausibly search for and saved them in ground_truth_keywords.xlsx (columns: file name, category, keywords).
3. **OCR and cleaning.** Each image goes through preprocessing, Tesseract and cleaning as described in Section 3.
4. **Keyword extraction.** TF-IDF is fitted on the 12 cleaned notices, while KeyBERT scores the candidate phrases in each notice on its own.
5. **Evaluation.** Both keyword lists are scored against the ground truth (Section 6).
6. **Demo.** A Gradio interface wraps the same functions so it can handle any new image.

Instructions for running

1. Open keyword_extraction_scanned_notices.ipynb in Google Colab (File → Upload notebook).
2. Click Runtime → Run all. The first cell installs Tesseract, pytesseract, KeyBERT, sentence-transformers and Gradio.
3. When the upload box appears, select all the notice images and ground_truth_keywords.xlsx together.
4. The notebook prints the OCR text, the keywords and the evaluation tables, and downloads keywords.csv, evaluation_results.csv, evaluation_summary.csv and f1_by_category.png.
5. The last cell starts the Gradio app and prints a public gradio.live link. Upload any scanned notice and click Submit.

Re-uploading files is safe. Colab renames duplicates to "name (1).jpg" and the notebook renames them back.

Practical applications

- **Searchable notice archives.** College websites could tag every scanned circular with keywords and let students search by topic.
- **Automatic routing.** Keywords such as *tuition fee* or *hall ticket* can send a notice to the accounts or examination section's feed.
- **Notifications.** Students could subscribe to a keyword (e.g. *scholarship*) and get alerted when a matching notice is posted.
- **Digitising paper records.** Offices with paper archives can scan and index them without typing anything out.

6. Results and Sample Outputs

Removing the letterhead and adding MMR raised KeyBERT's F1 from 0.261 to 0.571, which puts it close to TF-IDF (0.587). TF-IDF finds fewer of the true keywords but makes fewer mistakes. KeyBERT finds more of them but also includes more wrong ones.

Evaluation metric. For each notice, the predicted top-10 keywords and the ground-truth keywords are split into single words. Stopwords and digits are removed and the two word sets are compared:

- Precision = matched words / predicted words
- Recall = matched words / ground-truth words
- F1 = 2 × Precision × Recall / (Precision + Recall)

Scores are averaged over the 12 notices.

Overall results

Method	Setup	Precision	Recall	F1
TF-IDF	1–2 grams, full cleaned text	0.671	0.535	**0.587**
KeyBERT (baseline)	full text including letterhead, no MMR	0.238	0.292	0.261
KeyBERT (final)	letterhead removed, MMR diversity 0.3	0.502	0.676	**0.571**

The baseline KeyBERT result was the main problem we had to solve. On every notice, its top phrases were letterhead text such as *eastern institute*, *bhubaneswar odisha* and *eitm adm*. Because these lines repeat in every notice, the embedding model treats them as central to each document. TF-IDF avoids this, since inverse document frequency gives low weight to text found in every notice.

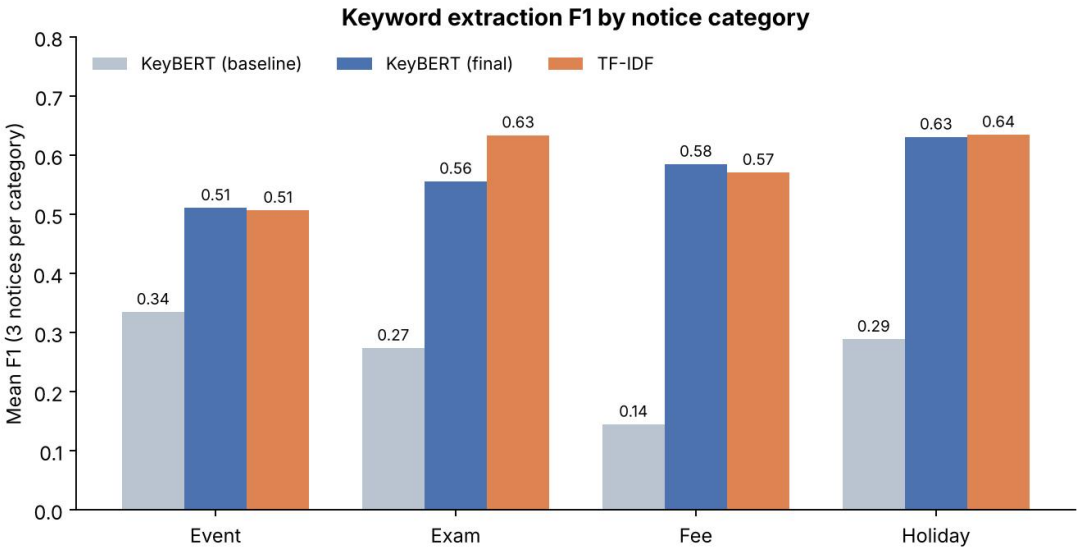


Figure 2: Mean F1 by notice category.

By category, TF-IDF is clearly better on exam notices (0.634 vs 0.556). KeyBERT is slightly ahead on fee notices (0.585 vs 0.571) and event notices (0.511 vs 0.507), and the two tie on holiday notices. Looking at individual notices, KeyBERT beats TF-IDF on 5 of the 12. F1 ranges from 0.30 (TF-IDF on notice10, the tech-fest notice) to 0.73 (TF-IDF on notice3, the practical exam schedule). Full per-notice scores are in evaluation_results.csv in the repository.

Sample output: notice1.jpg (End Semester Examination – Timetable)

	Keywords
Ground truth	end semester examination, timetable, 25 november 2026, hall ticket, exam fee, controller of examinations
TF-IDF	examination, hall, hall ticket, timetable, ticket, end, end semester, portal, tech, november
KeyBERT (baseline)	examinations eastern, bhubaneswar odisha, eastern institute, examination timetable, entry examination, examination hall, examinations tech, institute technology, eitm coe, examination section
KeyBERT (final)	examination timetable, semester examinations, examination hall, timetable available, end semester, november examination, examination section, examinations tech, college portal, tech programmes

The final KeyBERT output contains only phrases about the notice's content. TF-IDF returns overlapping fragments (*hall*, *ticket*, *hall ticket*) because each n-gram is scored separately.

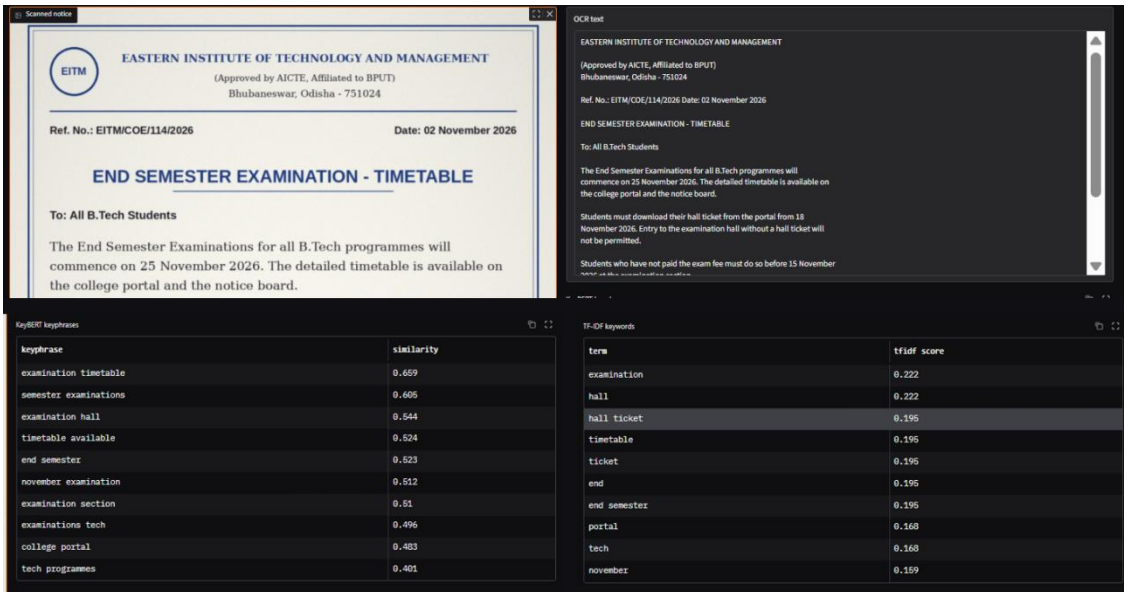


Figure 3: The Gradio demo on notice1.jpg: input scan, OCR text, KeyBERT keyphrases with similarity scores, and TF-IDF keywords with weights.

Sample output: notice10.jpg (TechNova 2026 – Annual Technical Fest)

	Keywords
Ground truth	technova 2026, technical fest, hackathon, registration, 5 december 2026, main auditorium, certificates
TF-IDF	technova, student activity, students department, main auditorium, main, open, awarded participants, awarded, auditorium events, auditorium
KeyBERT (final)	technova annual, presents technova, technova december, technova, technical fest, convenor technova, coding contest, hackathon robotics, computer science, activity centre

On this event notice KeyBERT matched more of the ground truth (*technova*, *technical fest*, *hackathon*, *december*) than TF-IDF (*technova*, *main auditorium*). The example also shows a weakness of each method:

- **KeyBERT repeats the dominant word.** 5 of its 10 phrases contain *technova*, so MMR at diversity 0.3 was not strong enough to spread the picks out. It also kept *convenor technova* from the signature line, which letterhead removal does not cover.
- **TF-IDF produces ties.** 9 of its 10 terms score exactly 0.113, because each one occurs once in this notice and in no other. Their order is arbitrary, so weak fragments such as *main*, *open* and *awarded* reach the top 10. Some bigrams, such as *students department*, never appear in the notice at all: scikit-learn removes stopwords before forming bigrams, so “students. The Department” becomes one term.

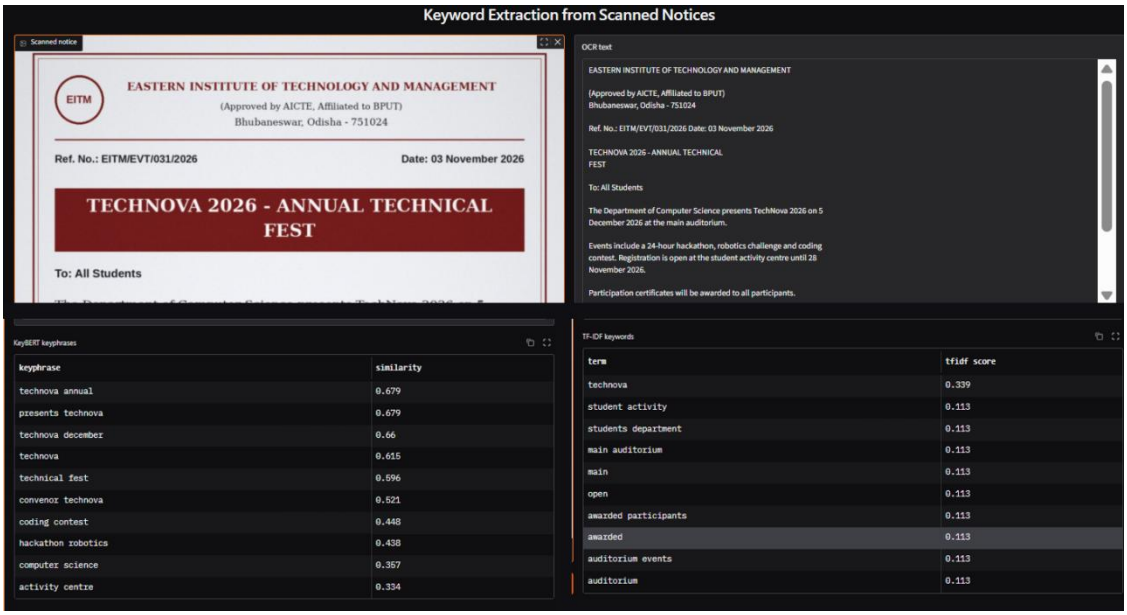


Figure 4: The Gradio demo on notice10.jpg (event notice).

Observations

- **OCR quality was high.** On these clean scans, Tesseract made only single-character errors, such as “Copy ta” for “Copy to” and “{Approved” for “(Approved”. Exact line matching missed these lines, which is why letterhead removal uses fuzzy matching.
- **The two methods complement each other.** KeyBERT gives readable phrases and has higher recall (0.676), while TF-IDF has higher precision (0.671).
- **Tuning was kept minimal on purpose.** Other TF-IDF settings changed F1 by only about ±0.04. With just 12 notices, tuning on the same data we evaluate on would overfit, so TF-IDF was left at its defaults.

Limitations

- **Small dataset:** 12 notices from one institute, so the scores show a trend rather than a statistically reliable result.
- **Clean scans:** the notices are clean, digitally produced images. Phone photos with skew, shadows or blur would cause more OCR errors, and those errors would carry into the keywords.
- **Dates and amounts are lost:** cleaning keeps only letters, so *25 november 2026* or *Rs. 18,500* can never become keywords, even though the ground truth includes them.
- **Lenient metric:** splitting phrases into words means *exam fee* partly matches *fee*, and month names (*november*) also count as hits.
- **One combined change:** letterhead removal and MMR were applied together, so we could not measure their separate effects on KeyBERT.
- **English and layout only:** Tesseract runs in English with automatic page segmentation, so Hindi or Odia notices, tables and multi-column layouts are not handled.
- **Demo vocabulary:** for a new notice, TF-IDF only knows words from the 12 training notices, while KeyBERT has no such limit.

7. Conclusion and Future Scope

The project delivers a working pipeline from scanned notice to keywords, along with a quantitative comparison and a web demo. The main finding is that a semantic model like KeyBERT needs document-specific boilerplate removed. Once the letterhead was removed, its F1 more than doubled (0.261 to 0.571) and came close to TF-IDF.

Achievements

- An end-to-end pipeline: OpenCV preprocessing, Tesseract OCR, fuzzy letterhead removal, TF-IDF and KeyBERT.
- A labelled evaluation set of 12 notices in 4 categories, with precision, recall and F1 reported per method and per category.
- A public Gradio app that extracts keywords from any uploaded notice.

Future scope

1. **Keep dates and amounts.** Add a regex or named-entity step (e.g. spaCy) for dates, fees and deadlines, which are often the most important facts in a notice.
2. **Combine the two methods.** Let TF-IDF pick candidate words and KeyBERT rank them, to get the precision of one and the recall of the other.
3. **Larger, real-world dataset.** Use 100+ notices, including phone photos and several institutes, and split them into tuning and test sets.
4. **Stronger OCR.** Add deskewing for photos and compare with TrOCR or EasyOCR on degraded scans.
5. **Indian languages.** Use Tesseract's Hindi and Odia models together with a multilingual embedding model (e.g. paraphrase-multilingual-MiniLM-L12-v2).
6. **LLM-based extraction.** Compare with an instruction-tuned LLM prompted to list keywords, scored with the same metric.

8. References and GitHub Repository

GitHub repository: https://github.com/i-karanburagohain/keyword-extraction-scanned-doc

Presentation video:
https://drive.google.com/file/d/1IqWBYf7Wyr79DL2a1_Tb96jweYh3c1wC/view?usp=sharing

References

1. Grootendorst, M. (2020). KeyBERT: Minimal keyword extraction with BERT. https://github.com/MaartenGr/KeyBERT
2. Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019. https://arxiv.org/abs/1908.10084
3. sentence-transformers/all-MiniLM-L6-v2. Hugging Face model card. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
4. Smith, R. (2007). An Overview of the Tesseract OCR Engine. ICDAR 2007. https://github.com/tesseract-ocr/tesseract
5. Otsu, N. (1979). A Threshold Selection Method from Gray-Level Histograms. IEEE Transactions on Systems, Man, and Cybernetics, 9(1), 62–66.
6. Buades, A., Coll, B. and Morel, J.-M. (2005). A non-local algorithm for image denoising. CVPR 2005.
7. Carbonell, J. and Goldstein, J. (1998). The use of MMR, diversity-based reranking for reordering documents and producing summaries. SIGIR 1998.
8. Salton, G. and Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513–523.
9. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12, 2825–2830. https://scikit-learn.org
10. Abid, A. et al. (2019). Gradio: Hassle-free sharing and testing of ML models in the wild. https://www.gradio.app

Dataset: 12 academic notice images and ground_truth_keywords.xlsx, included in the repository's data/ folder.